PrunePEFT: Iterative Hybrid Pruning for Parameter-Efficient Fine-tuning of LLMs

Anonymous Author(s)

Paper ID **512**

Abstract. Parameter Efficient Fine-Tuning (PEFT) methods have emerged as effective and promising approaches for fine-tuning pre-trained language models. Compared with Full parameter Fine-Tuning (FFT), PEFT achieves comparable task performance with a substantial reduction of trainable parameters, which largely saved the training and storage costs. However, using the PEFT method requires considering a vast design space, such as the type of PEFT modules and their insertion layers. Inadequate configurations can lead to sub-optimal results. Conventional solutions such as architectural search techniques, while effective, tend to introduce substantial additional overhead. In this paper, we propose a novel approach, PrunePEFT, which formulates the PEFT strategy search as a pruning problem and introduces a hybrid pruning strategy that capitalizes on the sensitivity of pruning methods to different PEFT modules. This method extends traditional pruning techniques by iteratively removing redundant or conflicting PEFT modules, thereby optimizing the fine-tuned configuration. By efficiently identifying the most relevant modules, our approach significantly reduces the computational burden typically associated with architectural search processes, making it a more scalable and efficient solution for fine-tuning large pre-trained models.

Keywords: Automated Fine-tuning · Neural Architecture Search · Deep Learning · Prune strategy.

1 Introduction

Parameter-efficient fine-tuning (PEFT) has emerged as a prominent paradigm for adapting pre-trained models to downstream tasks while preserving computational efficiency, particularly large language models (LLMs). These methods typically involve the introduction of compact adapter modules [11] [10], selective parameter freezing or tuning [38], or leveraging low-rank decomposition techniques to reduce the number of trainable parameters [13]. Notably, PEFT approaches can achieve performance comparable to full fine-tuning, despite training only a minuscule fraction of the full parameter set [11].

A myriad of PEFT methods have been proposed [13] [11] [39], all with the overarching objective of optimizing task performance while minimizing the parameter budget. However, the efficacy of these methods varies significantly across

different tasks, as they exhibit distinct sensitivities to task complexity and the allocated parameter budget [9]. Previous empirical investigations have also revealed that no single PEFT method universally outperforms others across all tasks [4], highlighting the necessity of tailoring the approach to specific task characteristics. Moreover, most existing PEFT applications are restricted to a singular method (e.g., Adapter, LoRA), leading to configurations that may not be optimal for a diverse array of tasks. Solving these challenges manually for each individual task is not only labor-intensive but also computationally inefficient. To address these limitations, researchers have developed versatile PEFT modules capable of adapting to specific tasks, such as AdaLoRA [39] and AdapterFusion [24], though the optimal performance and applicability of these modules are often non-intuitive.

In recent advancements, dynamic PEFT configuration strategies have garnered attention. [29] tackle this challenge by randomly dropping low-level adapters, resulting in only a marginal reduction in task performance. [40] and [14] extend this concept by integrating neural architecture search (NAS) and differentiable architecture search (DARTS)-based methods for fine-tuning strategy discovery, utilizing multi-objective Bayesian optimization to identify superior PEFT configurations. However, these methods typically suffer from high time complexity and significant computational overhead as the model depth and the number of PEFT modules increase. [16] integrates pruning strategies with NAS, but their approach is limited by a relatively constrained search space and a fixed pruning strategy, while still incurring substantial computational overhead.

To efficiently navigate the vast search space and identify the most effective fine-tuning configurations, we propose an adaptive hybrid pruning-based search strategy called PrunePEFT [1]. Given the immense parameter space of modern models, the introduction of additional PEFT modules represents a minimal increment in terms of overall parameter count. Our approach initializes the model by incorporating all potential PEFT modules across each layer and subsequently prunes less impactful modules. Drawing on a comprehensive review of pruning strategies [22], we introduce a hybrid pruning method that merges the advantages of multiple pruning strategies during the warm-up phase, effectively enhancing the quality of the search results. By leveraging this hybrid approach, we can progressively prune the candidate PEFT modules and identify the most efficient fine-tuned configurations under the specified parameter budget. Our contributions are summarized as follows:

New Problem Formulation: We formulate the PEFT strategy search as a structured pruning problem. We first construct a supernet by incorporating potential PEFT modules and then prune the supernet progressively. This formulation can effectively mitigate computational complexity while maintaining robust search performance.

Adaptive Hybrid Pruning Strategy: Based on the observation that different pruning strategies have specific tendencies towards different PEFT modules, we propose an adaptive hybrid pruning strategy to iteratively remove in-

[1] Code is available at https://anonymous.4open.science/r/prunePEFT

appropriate PEFT modules for each transformer layer, which can significantly enhance the performance compared to traditional methods relying on a single pruning approach.

SOTA Performance and Computational Efficiency.: Extensive experiments conducted on the multitasking benchmark GLUE [34] demonstrate the efficacy of our method. PrunePEFT successfully identifies fine-tuned configurations that match or surpass the performance of full fine-tuning, while only introducing additional overhead of approximately 30% training time.

2 Related Work

The related work can be categorized into three aspects: Parameter Efficient Fine-tuning, Automated PEFT, and pruning strategies. Our research shares objectives with PEFT, which aims to adapt pre-trained models to downstream tasks through selective and efficient parameter updates. Furthermore, balancing parameter efficiency with computational cost remains a key challenge in PEFT, which our work explicitly addresses. Automated fine-tuning constructs models through automated frameworks, with related works like [40], [14], and [15] optimizing the design of PEFT configurations using automated methods. In contrast, our pruning-based fine-tuning strategy employs multiple pruning strategies that have been proven effective in traditional machine learning [44]. We aim to demonstrate the effectiveness of these methods in the field of PEFT.

Parameter Efficient Fine-tuning. Due to massive trainable parameters and large storage requirements, [12] proposes parameter-efficient fine-tuning methods, where adapters enable transfer learning by adding additional bottleneck modules. [13] uses low-rank decomposition to reduce the number of parameters significantly. Many ingenious variants of adapters and LoRAs have emerged, such as adapter fusion [24], Mad-X [27], and AdaLoRA [39]. [21] mixes PEFT modules via a gating mechanism. [35] creates a mixture of Adapter modules. [19] decomposes pre-trained weights into magnitude and direction components. [10] unifies various adapter and LoRA variants, indicating that the current ingenious designs are combinations of specific components.

Automated PEFT. Automated machine learning (AutoML) has received increasing attention recently [15]. Recent efforts have explored combining PEFT with AutoML. [14] combines PEFT modules by probabilistic gating and differentiable search. [16] employs structured and unstructured pruning to learn PEFT configurations. [40] optimizes the model structure with Bayesian algorithm.

Pruning Strategies. Applying structured or unstructured pruning to neural networks is a well-established method. [36] introduces group-wise pruning on channels. [22] utilizes Taylor's formula as the pruning strategy. [7] and [20] hypothesize and try to verify the correctness of neural network pruning. [23] demonstrates the feasibility of the pruning strategy on NLP tasks and further discovers that

Table 1: Pruning strategies focus on information from multiple aspects of model computation.

Focus	Method	Standard	Formulas
FC	Weight	Mean squared magnitude of weights	$\Theta(m_i) = \frac{1}{\lvert \boldsymbol{w} \rvert} \sum_i w_i^2$
	Zeros	Proportion of non-zero weights	$\Theta(m_i) = \frac{1}{\lvert \boldsymbol{w} \rvert} \sum_i \mathbb{I}(w_i \neq 0)$
	Threshold	Proportion of weights below a threshold t	$\Theta(m_i) = \frac{1}{\lvert \boldsymbol{w} \rvert} \sum_i \mathbb{I}(\lvert w_i \rvert < t)$
IoF	Activation	Average activation magnitude	$\Theta(m_i) = \frac{1}{\lvert \boldsymbol{w} \rvert} \sum_i \lvert f(w_i) \rvert$
IoG	Sensitivity	Taylor expansion-based importance	$\Theta(m_i) = \frac{1}{\lvert \boldsymbol{w} \rvert} \sum_i \left\lvert w_i \frac{\partial \mathcal{L}}{\partial w_i} \right\rvert$
	Gradient	Mean absolute gradient magnitude	$\Theta(m_i) = \frac{1}{\lvert \boldsymbol{w} \rvert} \sum_i \left\lvert \frac{\partial \mathcal{L}}{\partial w_i} \right\rvert$

the winning tickets from large-scale datasets are applicable to small datasets. [2] prunes the Transformer model to reduce the number of parameters. [30] prunes 40% of the model's weights while having a minimal influence on performance. [31] compares post-pruning and pre-pruning and introduces early training epochs which are suitable for pruning.

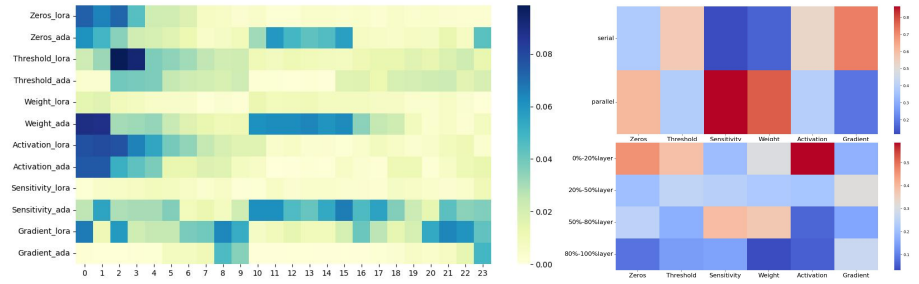
We adopt pruning strategies such as weight-based pruning [42], zero-count-based pruning [43], and Taylor-expansion-based pruning [22].

3 Motivation

PEFT Module Search as Network Pruning. Traditional approaches to optimizing sub-networks or hyperparameter configurations often involve search methodologies such as neural architecture search. These methods typically require multiple rounds of training and evaluation, resulting in substantial computational overhead and significant time consumption. While strategies like limiting the search time or employing low-fidelity optimization can enhance efficiency, the overall computational cost remains high. In contrast, pruning strategies iteratively shrink a super-network to identify optimal sub-networks. This approach inherently imposes an upper bound on computational resource consumption, which is substantially lower than direct search methods. Particularly in simplified search spaces, such as those for SA (serial adapter, e.g., Adapter) and PA (parallel adapter, e.g., LoRA) , if model contains n PEFT modules, pruning strategies can achieve time complexity as low as $O(n)$.

Prior research highlights that hidden layers in neural networks encode distinct types of information [33], with lower layers capturing general surface-level information, while deeper layers are more task-specific [32]. This allows for partitioning the model into distinct segments, enabling the application of varied PEFT methods across these partitions [1].

Pruning techniques leverage diverse criteria to identify and retain critical modules, including activation values, gradient information, and parameter-specific characteristics , etc [22]. These criteria correspond to different stages of model



(a) (b)

Fig. 1: The visualizations illustrate the sensitivities of distinct pruning strategies to the selection of PEFT modules. (a) depicts the pruning ratios of various strategies applied to different PEFT modules, with the horizontal axis representing layer indices. (b) summarizes the results: 1) The pruning tendencies of different strategies for various PEFT methods, and 2) The pruning tendencies of different strategies for PEFT modules in different model layers.

computation: feature characteristics (FC), information about the forward propagation stage (IoF), and information about gradients from backpropagation (IoG). Each of these strategies provides unique insights into model behavior, and their formulas are detailed in Table 1.

Our experimental findings corroborate previous research [8] that different pruning strategies yield inconsistent rankings of layer importance. Figure 1 highlights the pruning tendencies of various strategies, leading to the following observations:

Observation 1: Pruning strategies demonstrate PEFT method-specific preferences. Distinct pruning strategies show varying tendencies towards specific PEFT modules. For instance, the Taylor expansion-based strategy tends to preferentially prune the serial modules within the PEFT framework, emphasizing its sensitivity to PEFT configurations rather than solely to depth.

Observation 2: Pruning strategies exhibit varying sensitivities to model depth. Pruning strategies display distinct preferences for layers at different depths within the model. For example, in tasks derived from the GLUE dataset, the activation-based pruning strategy primarily targets layers located in the top 20% of the model, suggesting a stronger sensitivity to upper-layer representations.

Each pruning strategy selects and removes less significant components of the model based on specific, valid information sources to enhance performance. To further improve the quality of the searched fine-tuned structures, we propose a unified hybrid pruning strategy that organically integrates insights from these diverse information sources.

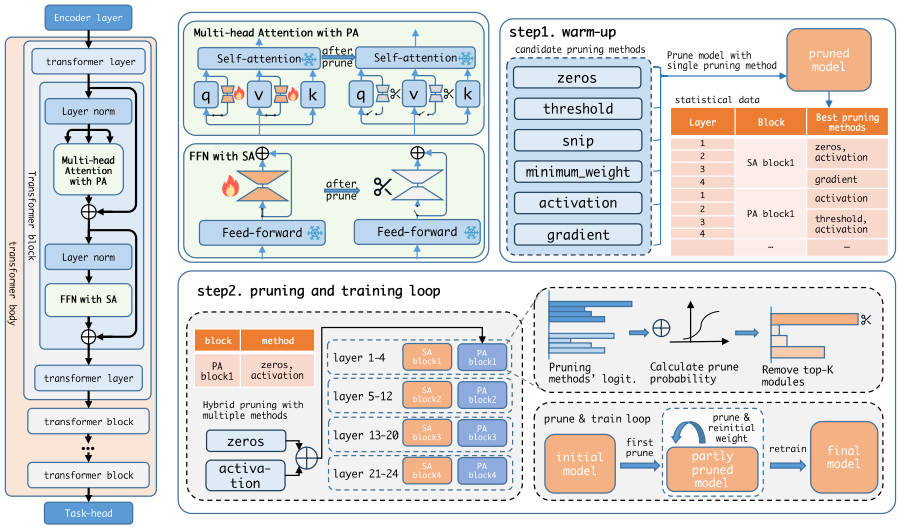


Fig. 2: The overall framework of the proposed PrunePEFT (i.e., iterative hybrid pruning), which consists two phases: warm-up phase and iterative pruning phase.

4 The Proposed Method: PrunePEFT

The goal of this work is to identify the optimal fine-tuning configuration M under the constraint of a predefined and finite budget B of trainable parameters. In this paper, we propose a novel iterative hybrid pruning approach called PrunePEFT to eliminate less important PEFT modules. As shown in Figure 2, the overall framework consists of two primary phases: (i) warm-up phase that determines the set of hybrid pruning strategies for each mode partition, and (ii) iterative pruning phase that achieves iterative hybrid pruning tailored to downstream tasks. After multiple rounds of pruning, we retrain the searched fine-tuned configuration. The proposed method enables both the architectural search and the control of sparsity in PEFT modules.

4.1 Iterative Hybrid Pruning

Hybrid Pruning Strategy Selection with Warm-up. To identify the most critical PEFT modules, we select pruning strategies based on various sources of information as outlined in Table 1, thereby employing a diverse set of pruning strategies.

The model is partitioned into four spindle-shaped blocks [1], with a warm-up phase introduced prior to training. During the warm-up phase, the original dataset D is sampled to generate a compact dataset d, which maintains the original label distribution. The model then undergoes multiple rounds of heuristic training on d. Throughout these training iterations, we track the pruning

frequency of each pruning strategy for various PEFT modules. This pruning frequency represents the pruning tendencies for each strategy. Based on these observed tendencies, we assign the pruning strategy S_i that most effectively matches the characteristics of partition P_i. The pruning strategy S_i of each partition forms the hybrid pruning strategy S_H.

For partition P_i, we employ Bayesian model averaging to jointly determine the pruned layers. Specifically, the probability $p(m_i)$ of pruning module m_i is computed as follows:

$$\begin{aligned} p(m_i) &= \frac{e^{k_i \cdot \Theta(m_i, S_i)}}{\sum_{j=1}^{n} e^{k_j \cdot \Theta(m_j, S_j)}}, \\ \text{where} \quad k_i &= \text{Sigmoid}(rank(m_i, S_i)). \end{aligned} \tag{1}$$

Here, $\Theta(m_i, S_i)$ represents the evaluation metric of PEFT module m_i under pruning strategy S_i, and k_i is the weight determined by the rank of the module's likelihood of being pruned by strategy S_i. The hybrid pruning strategy S_H for the entire dataset D is derived by combining the pruning strategies from all partitions.

Iterative Pruning. To obtain the fine-tuned configuration M, we apply iterative pruning approach based on hybrid pruning strategy S_H identified during the warm-up phase. In each round of pruning, the model is trained in several steps using either the full dataset D or the trimmed dataset d, with both forward and backward propagation data (e.g., gradients) recorded. During each pruning round, the hybrid pruning strategy S_H computes the probability of pruning each module based on intermediate information. The modules with the highest k pruning probabilities are removed from the model:

$$\text{mask} = \text{Onehot}(TopK(P, k)) \tag{2}$$

$$M_i = \text{mask}(M_{i-1}) \tag{3}$$

where P denotes the importance ranking based on the hybrid pruning strategy S_H, and M_i represents the model after the i-th round of pruning. Each subsequent round of pruning operates on the model M_i obtained from the previous iteration. If the current round is not the last, the pruned PEFT modules are reinitialized as follows [16]:

$$W_{down} \sim \mathcal{N}(0, 1/\sqrt{d_{down}}), \tag{4}$$

$$W_{up} \sim \mathcal{N}(0, 1/\sqrt{d_{up}}), \tag{5}$$

where W_{down} and W_{up} correspond to the down-projection and up-projection matrices, respectively, and d_{down} and d_{up} denote their corresponding dimensions.

After completing the predetermined number of pruning rounds, the total number of trainable parameters reaches the target budget B. At this point, the remaining PEFT modules are considered the final fine-tuned configuration M^*

Algorithm 1 Iterative Hybrid Pruning

Input: Dataset D, initial model M with all PEFT modules, number of pruning rounds r
Output: Final pruned model M^*
Warm-up Phase:
Train the model on the trimmed dataset $d \subseteq D$. For each partition P_i of the model, calculate the pruning strategy S_i using heuristic methods. Based on the observed tendencies, assign the appropriate pruning strategy to each partition to form the hybrid pruning strategy S_H.
Iterative Pruning Phase:
for $i = 1$ **to** `r` **do**
    Compute the pruning probability vector P for each module using the hybrid pruning strategy $\Theta(M_{i-1}, S_H)$.
    Identify and remove the top k layers based on the pruning probability P: mask = Onehot(TopK(P, k)). Update the model: $M_i = \text{mask}(M_{i-1})$.
    Reinitialize the pruned layers: $W_{down} \sim \mathcal{N}(0, 1/\sqrt{d_{down}})$, $W_{up} \sim \mathcal{N}(0, 1/\sqrt{d_{up}})$.
end for
Return: The pruned model M^* after r rounds, representing the final configuration.

for the task at hand. The output of layer h_i is then computed as:

$$H_{\text{out}} = \text{mask}_{h_i}^{SA} \cdot \left(\sigma(H_{\text{in}} \cdot W_{\text{down}}^{SA}) \cdot W_{\text{up}}^{SA}\right) + \text{mask}_{h_i}^{PA} \cdot \left(\sigma(H_{\text{in}} \cdot W_{\text{down}}^{PA} \cdot W_{\text{up}}^{PA})\right) \tag{6}$$

where $\text{mask}_{h_i}^{SA}$ and $\text{mask}_{h_i}^{PA}$ belonging to the mask_{h_i} are boolean indicators that control whether the corresponding PEFT modules are activated at layer h_i. $\sigma(\cdot)$ is the nonlinear activation function such as ReLU or GELU. The overall workflow of iterative hybrid pruning is shown in Algorithm 1.

4.2 Complexity Analysis

For our search space, the total number of possible configurations is given by $(N_{PA} \cdot N_{SA})^{|H|}$, where N_{PA} and N_{SA} represent the number of choices for PA and SA methods at each layer, respectively, and $|H|$ denotes the number of layers in the model. During a forward search, evaluating each configuration requires r epochs, with each epoch taking time t, leading to substantial computational overhead. PrunePEFT mitigates this challenge by using the pruning approach. In each pruning round, after one training epoch to gather necessary information, k modules of lower importance are pruned. Given that, in the worst case, all of PEFT modules in model may be eliminated, the total time T_{search} overhead for evaluating a configuration is bounded by:

$$T_{search} = (N_{PA} \cdot N_{SA}) \cdot (|H|/k) \cdot t. \tag{7}$$

This approach significantly reduces the search time overhead, making it particularly advantageous for search spaces with a larger number of layers and fine-tuning modules.

Table 2: **Results on GLUE Benchmark with RoBERTa-large.** We present the best fine-tuning performance for PrunePEFT and baselines across each GLUE task. The parameter percentage denotes the ratio of additional parameters introduced by fine-tuning relative to the number of parameters in the pre-trained model. Avg-R shows the average rank of each method on all tasks. We replicate some of baseline configurations, with the best performance highlighted in **bold** and the sub-optimal result highlighted in underline.∗ denotes the results that are directly copied from Hugging face and S-MaM [16].

Method	**%Param.**	**MNLI**	**QNLI**	**QQP**	**RTE**	**SST-2**	**MRPC**	**CoLA**	**Avg-Rank**
FFT*	100%	90.2	94.7	**92.2**	86.6	96.4	90.9	68.0	4.7
Manual PEFT Methods									
Adapter(r=128)	2%	90.2	94.7	90.9	85.3	96.1	90.4	66.6	8.0
LoRA(r=32)	1%	89.9	94.8	91.2	85.2	95.7	90.2	67.0	8.6
AdaMix(r=32)	1%	90.3	94.4	90.4	87.4	96.6	90.9	**69.9**	5.1
AdaLoRA(r=32)	1%	90.2	94.8	91.0	**88.1**	96.3	90.5	66.3	6.1
DoRA(r=32)	1%	90.1	94.7	91.3	87.0	95.7	**91.4**	66.2	7.1
Automated PEFT Methods									
S3Delta	1%	90.3	**94.9**	91.4	86.3	96.1	90.7	66.8	5.3
S-MaM*	1%	**90.6**	94.5	90.6	85.2	95.9	90.4	66.3	8.3
AutoPEFT	1%	90.5	**94.9**	91.8	85.6	96.3	90.7	69.2	3.7
PrunePEFT	1%	90.5	**94.9**	91.8	87.7	**96.9**	**91.4**	68.3	**1.7**
PrunePEFT(low fidelity)	1%	90.3	**94.9**	91.5	87.3	96.6	91.2	67.7	3.3

5 Experiments

5.1 Experimental Settings

Datasets and Pre-trained Models (PTMs) To comprehensively evaluate the performance and generalizability of PrunePEFT, we conduct extensive experiments across diverse model scales and task domains. Consistent with established PEFT benchmarks, we first evaluate our method on the GLUE benchmark [34] using RoBERTa-large [18] (326M parameters) as the representative encoder-based backbone. To further validate the scalability of our approach to large language models (LLMs) and complex generative reasoning, we extend our evaluation to the Llama-2-7b model [47] (7B parameters) using the GSM8K [45] and HumanEval [46] datasets. All datasets are sourced from the Hugging Face Datasets repository [17].

Baselines We evaluate our adaptive hybrid pruning strategy against three categories of baselines.

Full Fine-Tuning (FFT). The traditional approach, where all parameters of the pre-trained model are fine-tuned during training.

Manual PEFT Methods. This category includes some representative PEFT methods, such as LoRA [13] and Adapters [11], which have been widely adopted as standard baselines. The LoRA approach is applied to the query and value

matrix of the attention layers following the configuration described in [16]. In addition, we also consider several recent variants, including DoRA [19], AdaLoRA [39] and AdaMix [35], to verify the validity of PrunePEFT.

Automated PEFT Methods. To further contextualize the performance of our method, we compare it with several existing automated fine-tuning strategies, including AutoPEFT [40], S3Delta [14], and S-MaM [16]. For fairness, we ensure that search space and parameter budget of S3Delta and AutoPEFT are aligned with those used in PrunePEFT.

Implementation Details We implement our framework utilizing the codebases from AdapterHub [25] and the Hugging Face PEFT library [26]. For experiments on RoBERTa-large, we employ batch sizes of 16 or 32, with learning rates swept within the range of $(10^{-5}, 10^{-3})$ to optimize performance across the GLUE benchmark. For complex reasoning and generation tasks on Llama-2-7b, we fine-tune the model for 1 epoch with an effective batch size of 128. In all configurations, we conduct extensive hyperparameter tuning to ensure a fair comparison between our proposed PrunePEFT and the baseline methods.

5.2 Results on GLUE

Table 2 summarizes the performance of our method on various tasks from the development set of GLUE benchmark using RoBERTa-large. It reveals that among manual PEFT methods, none is able to consistently achieve superior performance across all benchmarks.

Compared to the PEFT methods encompassed within the search space of PrunePEFT, our approach demonstrates improvements across all tasks. Moreover, when compared against other manual PEFT variants as well as FFT, PrunePEFT consistently achieves superior performance on multiple tasks and in terms of overall average performance. When compared to other automated PEFT methods under an equivalent parameter budget, PrunePEFT also attains the highest average score. Results demonstrates that using basic fine-tuning modules and automatically searching both their combinations and per-layer assignments can surpass the performance of existing complex SOTA methods, thereby demonstrating the value of automated fine-tuning strategy search.

Furthermore, we introduce a low-fidelity variant of PrunePEFT, which performs pruning using only a small subset (approximately 1%) of the original dataset. This significantly reduces the overhead during the search phase, while still surpassing the performance of FFT.

5.3 Results on Large Language Models

Table 3 summarizes the performance of our method on complex mathematical reasoning and code generation tasks, utilizing the GSM8K and HumanEval datasets. It reveals that among manual PEFT methods, none is able to consistently achieve superior performance across different domains; for instance,

Table 3: Performance comparison of various fine-tuning methods on the GSM8K and HumanEval datasets. FFT denotes Full Fine-Tuning. The best overall results are in **bold**, and the best results among PEFT methods are underlined.

Method	GSM8K (%)	HumanEval (%)
FFT	**35.50**	19.87
Adapter	31.46	19.51
LoRA	33.97	17.07
DoRA	33.93	25.61
PrunePEFT (Ours)	34.72	**27.44**

DoRA performs exceptionally well on HumanEval but is slightly outperformed by LoRA on GSM8K.

Compared to the PEFT methods encompassed within the search space, our approach demonstrates significant improvements across both tasks. Moreover, when compared against other manual PEFT variants as well as full fine-tuning (FFT), PrunePEFT consistently achieves superior performance on the HumanEval benchmark (27.44%), outperforming existing methods. On the GSM8K dataset, PrunePEFT attains the highest score among all parameter-efficient methods (34.72%), closely rivaling the performance of FFT (35.50%). Results demonstrate that automatically searching for optimal fine-tuning configurations can effectively bridge or even surpass the gap with full fine-tuning on complex reasoning and generation tasks, thereby demonstrating the value of automated fine-tuning strategy search.

Other automated fine-tuning methods often introduce prohibitive computational overhead when scaled to large language models (as detailed in Section 5.4), so we refrain from evaluating them on Llama-2 due to these practical constraints. In contrast, our proposed PrunePEFT maintains high efficiency, incurring an additional overhead of less than 1% on both GSM8K and HumanEval, further highlighting its suitability for LLM optimization.

5.4 Efficiency of the Search Process

Traditional architectural search methods typically incur significant time overhead when searching for optimal configurations. To assess the efficiency of our approach, we analyze the ratio of search epochs to retraining epochs during the fine-tuned architecture search on RoBERTa-large.

As shown in Table 4, PrunePEFT introduces an additional overhead of less than 30% of the retraining time for fine-tuned architecture search including the time of the warm-up phase, while PrunePEFT (low fidelity) incurs only 1% of the retraining time. This is significantly lower than AutoPEFT (9.25× retraining) or S3Delta (6.5× retraining). This advantage becomes particularly pronounced when fine-tuning models with larger parameter counts.

Table 4: The average ratio of search time to retraining time for the fine-tuned architecture search.

Method	**Average epochs**		
	Search	**Retrain**	**Ratio**
LoRA	0	20	NaN
AutoPEFT	185	20	9.25
S3Delta	130	20	6.50
S-MaM	20	20	1.00
PrunePEFT	6.0	20	**0.30**
PrunePEFT (low fidelity)	0.1	20	**0.01**

Table 5: Transferability evaluation of PrunePEFT with different configurations on RoBERTa-large across the GLUE benchmark.

Source Task	**Transfer Type**	**MRPC**	**SST-2**	**MNLI**
RTE	Strategy	91.2	96.6	90.3
RTE	Architecture	90.9	96.4	90.1
CoLA	Strategy	91.3	96.4	90.3
CoLA	Architecture	91.0	96.3	90.3
No Transfer	NaN	91.4	96.9	90.5

5.5 Transferability

We assess the transferability of the PrunePEFT method by using task RTE and CoLA as the source dataset. We evaluate the transferability of both the hybrid pruning strategy determined during the warm-up phase, and the fine-tuned architecture derived from the search in the source dataset. The strategies and configurations are then applied to other datasets for further training.

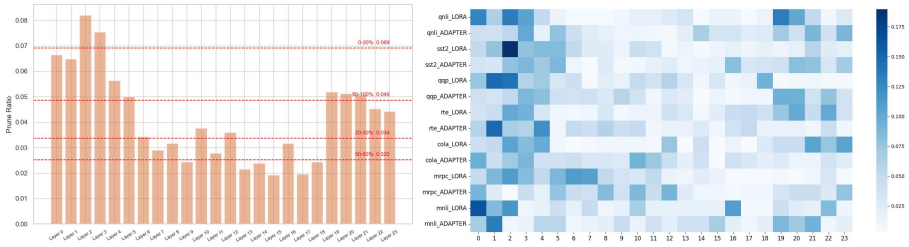
As presented in Table 5, the strategy and architecture from source dataset exhibits strong transferability, whose performance closely matching the performance achieved on the original task. This confirms the reusability of both the hybrid pruning strategy and the fine-tuned architecture, demonstrating the method's adaptability across different datasets.

5.6 Ablation Study

We conduct a series of ablation studies to evaluate the effectiveness of the hybrid pruning strategy for structural optimization. The experiments include: a no-pruning baseline, random pruning, single pruning strategy (selected from Table 1), and hybrid pruning without block partition. Since the optimal single pruning strategy varies across tasks, we report the best-performing result for each task. As shown in Table 6, relative to other strategies, the hybrid approach yields more effective model configurations at the search process by extracting richer task-relevant information, resulting in superior performance and stability.

Table 6: Ablation study with different pruning strategies with the RoBERTa-large model under the GLUE benchmark .

Method	MNLI	QNLI	QQP	RTE	SST-2	MRPC	CoLA	Avg
Without Prune	90.0	94.5	90.8	84.8	96.1	90.5	66.8	87.6
Random Prune	90.0	94.3	90.9	85.1	95.9	90.3	67.0	87.6
Prune with Single Strategy	90.3	94.7	91.3	86.6	96.4	90.9	67.3	88.2
Without Block Partition	90.3	94.8	91.6	87.3	96.6	91.1	67.9	88.5
PrunePEFT	**90.5**	**94.9**	**91.8**	**87.7**	**96.9**	**91.4**	**68.3**	**88.8**



(a) (b)

Fig. 3: Visualization of the fine-tuned configurations obtained through PrunePEFT. (a) presents the frequency that PEFT module is pruned in diffrent layers and the average frequency in four blocks. (b) presents the frequency with which PEFT module is pruned across different downstream tasks.

5.7 Explanations of the Searched Structures

We conduct an analysis of the fine-tuned configurations identified by PrunePEFT and deliberate on the most effective locations for placing PEFT modules within the network. Figure 3 (a) visualizes the fine-tuned configurations obtained through PrunePEFT. The PEFT modules deemed unimportant predominantly appear in the initial layers of the model, with a smaller proportion found in the deeper layers and few scattered in intermediate positions. This suggests that fine-tuning layers situated in the middle of the model may yield more effective results.

Regarding the selection of PEFT modules, Figure 3 (b) reveals a staggered distribution in the frequency with which different PEFT methods are pruned. Specifically, layers where LoRA modules are applied tend to be unsuitable for incorporating Adapter modules, and vice versa. This finding underscores that a single fine-tuning method is not universally applicable across all layers of the model. Consequently, it highlights the importance of performing an architectural search to identify the most suitable configuration for each layer, ensuring optimal model performance.

6 Conclusion and Future Work

In this paper, we proposed a novel method for fine-tuning configuration search based on adaptive hybrid pruning, termed PrunePEFT. Experimental results demonstrate that PrunePEFT effectively removes redundant structures, enabling the identification of the optimal PEFT configuration while incurring significantly lower time overhead compared to traditional configuration search methods. There remain several avenues for future work: (1) Exploring more efficient search spaces to enhance the potential of configuration search, and (2) Extending the current pruning strategy to incorporate unstructured pruning techniques, and investigating its impact on fine-tuned configuration search.

References

1. Chen, J., Zhang, A., Shi, X., Li, M., Smola, A., Yang, D.: Parameter-Efficient Fine-Tuning Design Spaces. arXiv preprint arXiv:2301.01821 (2023)
2. Cheong, R.: transformers . zip : Compressing Transformers with Pruning and Quantization. In: 1 (2019). https://api.semanticscholar.org/CorpusID:201694931
3. Devlin, J.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018)
4. Ding, N., Qian, Y.-j., Yang, G., Wei, F.-c., Su, Y.-s.: Parameter-efficient fine-tuning of large-scale pre-trained language models. Nature machine intelligence (2022)
5. Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C.-M., Chen, W. et al.: Parameter-efficient fine-tuning of large-scale pre-trained language models. Nature Machine Intelligence **5**(3), 220–235 (2023)
6. Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A. et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
7. Frankle, J., Carbin, M.: The lottery ticket hypothesis: Finding sparse, trainable neural networks. arXiv preprint arXiv:1803.03635 (2018)
8. He, W., Huang, Z., Liang, M., Liang, S., Yang, H.: Blending pruning criteria for convolutional neural networks. In: Artificial Neural Networks and Machine Learning–ICANN 2021: 30th International Conference on Artificial Neural Networks, Bratislava, Slovakia, September 14–17, 2021, Proceedings, Part IV 30, pp. 3–15. Springer (2021)
9. He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., Neubig, G.: Towards a unified view of parameter-efficient transfer learning. arXiv preprint arXiv:2110.04366 (2021)
10. He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., Neubig, G.: TOWARDS A UNIFIED VIEW OF PARAMETER-EFFICIENT TRANSFER LEARNING. In: Proceedings of the International Conference on Learning Representations (2022)
11. Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S.: Parameter-Efficient Transfer Learning for NLP. CoRR (2019)
12. Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S.: Parameter-efficient transfer learning for NLP. In: International conference on machine learning, pp. 2790–2799. PMLR (2019)
13. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 (2021)

14. Hu, S., Zhang, Z., Ding, N., Wang, Y., Wang, Y., Liu, Z., Sun, M.: Sparse structure search for delta tuning. Advances in Neural Information Processing Systems **35**, 9853–9865 (2022)
15. Jin, H., Song, Q., Hu, X.: Auto-keras: An efficient neural architecture search system. In: Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, pp. 1946–1956 (2019)
16. Lawton, N., Kumar, A., Thattai, G., Galstyan, A., Steeg, G.V.: Neural architecture search for parameter-efficient fine-tuning of large pre-trained language models. arXiv preprint arXiv:2305.16597 (2023)
17. Lhoest, Q., Del Moral, A.V., Jernite, Y., Thakur, A., Von Platen, P., Patil, S., Chaumond, J., Drame, M., Plu, J., Tunstall, L. et al.: Datasets: A community library for natural language processing. arXiv preprint arXiv:2109.02846 (2021)
18. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv preprint arXiv:1907.11692 (2019). https://arxiv.org/abs/1907.11692
19. Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C.F., Cheng, K.-T., Chen, M.-H.: DoRA: Weight-Decomposed Low-Rank Adaptation. ICML 2024 (2024)
20. Malach, E., Yehudai, G., Shalev-Schwartz, S., Shamir, O.: Proving the lottery ticket hypothesis: Pruning is all you need. In: International Conference on Machine Learning, pp. 6682–6691. PMLR (2020)
21. Mao, Y., Mathias, L., Hou, R., Almahairi, A., Ma, H., Han, J., Yih, W.-t., Khabsa, M.: Unipelt: A unified framework for parameter-efficient language model tuning. arXiv preprint arXiv:2110.07577 (2021)
22. Molchanov, P., Tyree, S., Karras, T., Aila, T., Kautz, J.: Pruning convolutional neural networks for resource efficient inference. arXiv preprint arXiv:1611.06440 (2016)
23. Morcos, A., Yu, H., Paganini, M., Tian, Y.: One ticket to win them all: generalizing lottery ticket initializations across datasets and optimizers. Advances in neural information processing systems **32** (2019)
24. Pfeiffer, J., Kamath, A., Rücklé, A., Cho, K., Gurevych, I.: Adapterfusion: Non-destructive task composition for transfer learning. arXiv preprint arXiv:2005.00247 (2020)
25. Pfeiffer, J., Rücklé, A., Poth, C., Kamath, A., Vulić, I., Ruder, S., Cho, K., Gurevych, I.: Adapterhub: A framework for adapting transformers. arXiv preprint arXiv:2007.07779 (2020)
26. Mangrulkar, S., Paul, S., Sanh, V., Gugger, S.: PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. GitHub repository (2022). https://github.com/huggingface/peft
27. Pfeiffer, J., Vulić, I., Gurevych, I., Ruder, S.: Mad-x: An adapter-based framework for multi-task cross-lingual transfer. arXiv preprint arXiv:2005.00052 (2020)
28. Ren, P., Xiao, Y., Chang, X., Huang, P.-Y., Li, Z., Chen, X., Wang, X.: A comprehensive survey of neural architecture search: Challenges and solutions. ACM Computing Surveys (CSUR) **54**(4), 1–34 (2021)
29. Rücklé, A., Geigle, G., Glockner, M., Beck, T., Pfeiffer, J., Reimers, N., Gurevych, I.: Adapterdrop: On the efficiency of adapters in transformers. arXiv preprint arXiv:2010.11918 (2020)
30. Sajjad, H., Dalvi, F., Durrani, N., Nakov, P.: Poor man's bert: Smaller and faster transformer models. arXiv preprint arXiv:2004.03844 **2**(2) (2020)
31. Shen, M., Molchanov, P., Yin, H., Alvarez, J.M.: When to prune? a policy towards early structural pruning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 12247–12256 (2022)

32. Tenney, I.: BERT rediscovers the classical NLP pipeline. arXiv preprint arXiv:1905.05950 (2019)
33. Vulić, I., Ponti, E.M., Litschko, R., Glavaš, G., Korhonen, A.: Probing pretrained language models for lexical semantics. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 7222–7240 (2020)
34. Wang, A.: Glue: A multi-task benchmark and analysis platform for natural language understanding. ICLR (2019)
35. Wang, Y., Agarwal, S., Mukherjee, S., Liu, X., Gao, J., Awadallah, A.H., Gao, J.: AdaMix: Mixture-of-adaptations for parameter-efficient model tuning. arXiv preprint arXiv:2205.12410 (2022)
36. Wen, W., Wu, C., Wang, Y., Chen, Y., Li, H.: Learning structured sparsity in deep neural networks. Advances in neural information processing systems **29** (2016)
37. Yu, R., Li, A., Chen, C.-F., Lai, J.-H., Morariu, V.I., Han, X., Gao, M., Lin, C.-Y., Davis, L.S.: Nisp: Pruning networks using neuron importance score propagation. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 9194–9203 (2018)
38. Zaken, E.B., Ravfogel, S., Goldberg, Y.: Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. arXiv preprint arXiv:2106.10199 (2021)
39. Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., Zhao, T.: AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. arXiv preprint arXiv:2303.10512 (2023)
40. Zhou, H., Wan, X., Vulić, I., Korhonen, A.: Autopeft: Automatic configuration search for parameter-efficient fine-tuning. Transactions of the Association for Computational Linguistics **12**, 525–542 (2024)
41. Zoph, B.: Neural architecture search with reinforcement learning. arXiv preprint arXiv:1611.01578 (2016)
42. Han, S., Pool, J., Tran, J., Dally, W.: Learning both weights and connections for efficient neural network. Advances in neural information processing systems **28** (2015)
43. Hu, H.: Network trimming: A data-driven neuron pruning approach towards efficient deep architectures. arXiv preprint arXiv:1607.03250 (2016)
44. Vadera, S., Ameen, S.: Methods for pruning deep neural networks. IEEE Access **10**, 63280–63300 (2022)
45. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R. et al.: Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168 (2021)
46. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G. et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
47. Touvron, H., Martin, L., Kevin, R., Xavier, M., Baptiste, R., Gautier, I., Thomas, S., Edouard, G., Thibaut, L., Guillaume, L. et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)